The Thiele Machine

A Mathematical Reference

Devon Thiele

Before the specification. I don't do math by hand. I think in shapes, directions, and connections, and my part of this work was to see where things had to fit, to direct, and to refuse what didn't fit. This document is the translation. Language models wrote the formal statements and the proofs, and I read the statements to make sure they say what the shape says. Coq checks the formal counterparts identified below; some on-page arguments have no Coq counterpart here, as noted where they occur. The arguments that carry the results are in the monograph, on the page; the formal versions here are their counterparts, and compiling them is a tool for catching mistakes, not the reason to believe them. So nothing below is in my voice, on purpose. It's the reference layer: exact statements, exact premises, and where each one lives. If you want the pictures, the monograph starts with them. If you want to know exactly what is claimed, it's here.

The Thiele Machine is an abstract model of computation, and an argument made with it about what an account of computation should preserve.

It is a machine in the sense a Turing machine is one: a mathematical object. No device is meant. The model is a set of states, moves, a step, a price on each move, a record that can be compared, and one rule, the toll: a move that takes the record out of the down-set of where it stood pays at least one. Any system with those parts is a Thiele machine. A yes-or-no reading is the smallest record, and most of what follows is stated on it. Forget the prices and the record and what is left is an ordinary deterministic machine; the results below are about what that forgetting throws away. A small two-counter machine whose commitments are earned is one example, and a universal machine U is another. They show the definition can be met. They are not the model.

The structural question is what a total deterministic process needs to raise a reading and keep it: a merge along the run or endlessly fresh states. A physical price needs separate hypotheses. Landauer bounds depend on the physical model and input distribution; finite-time entropy-production bounds here concern specified thermal stochastic dynamics, not every physical state change. Certification, a record that a claim was checked and closed, is the sharpest case. It is one example of a wider question about structural state, observation and cost. A window, a function from states to what can be seen of them (Chapter 5), can lose distinctions that the full state keeps, such as the record and the ledger; the cost laws of Chapters 3 and 4 constrain specified events under hypotheses each theorem names. A conventional encoding can preserve those distinctions. A window that discards them cannot recover them in general. The stronger proposal that these distinctions belong in a necessary foundation of computation is a conjecture. The status of each result is stated below.

Abstract

This is a mathematical reference for the Thiele Machine: a deterministic transition system with instruction prices and a record that is a position in a preorder, with a toll on every step that moves the record out of the down-set of where it stood. The one-bit record, a Boolean reading with its false-to-true crossing, is the two-point instance, and most results are stated on it. Permanence and an internal ledger are additional conditions used by particular results. It states the model, its toll and the universal floor, then the results that hold for every machine meeting it, and gives one small example machine to show the definition can be met. Each result is stated with its premises and names the declaration that carries it.

A formal proof shows that conclusions follow from premises. It does *not* show that premises model physical reality. Every result is tagged as one of three types: **(S)** structural theorems about the model or a stated machine, **(C)** conditional results that hold under a named premise no machine fact discharges, and **(R)** refutations, where a theorem states the negation of a proposed claim and a named construction supplies the counterexample. One proposal, the pointer criterion of Chapter 13, has no proof; it carries a fourth tag, thesis.

The structural results (S), by where they sit in the model:

- *The definition and the price in the move.* The floor is the toll over any order of records, and the universal floor on the one-bit instance. The least instruction price and the least price for the lift. On finite machines, the toll derived from merge pricing for permanent certificates. Shadow pricing: no price computed from a window that hides a crossing is exact.

- *What stays.* A free move cannot touch a permanent reading. The record axis over any base is a latch, and a growing record decomposes into threshold latches. The small machine's earned provenance, checker soundness and minimum price.

- *What a window forgets.* The coarsest observational identification and the terminal lift. The window theorem for every Thiele-complete machine, with a gap of one, and for every certified Thiele-complete machine, with a gap of three.

- *The room a kept distinction needs.* On any state space a permanent flip merges or needs a fresh state every time, with the exact retention bound for a bounded memory, and the stochastic and unitary forms of the same count.

- *Computation as the special case.* Thiele completeness and certified Thiele completeness; the lift of every universal base to a certified Thiele-complete machine and its converse; composition; the universal machine U and the priced host's program $\mathrm{U_P}$; a verified compiler onto these machines; the recursion theorem and Rice's theorem (the logician Henry Rice, 1953: no program decides a property of what programs do, unless it holds of every program or of none) for L, and Rice's theorem for the two-counter machine itself.

- *The world.* The classical CHSH bound; the algebraic Tsirelson bound; and, on the zero-marginal slice, the equivalence of column contractivity with positive semidefiniteness of one fixed 5×5 NPA matrix.

The failure of every clock-style machine, the limits of lifting and of composition, and the necessity of each clause of Thiele completeness are refutations (R). Results stated over Landauer heat are conditional (C) on the named premise `landauer_heat`.

Contents

1 Introduction **3**

2 Notation and Conventions **4**

3 Thiele Machines and the Universal Floor **5**
3.1 Free moves, least prices and the observable lift 6
3.2 A2 among cost frameworks 9
3.3 Several floors and one charge 10

4 Finite Machines and the Price of Merging **11**
4.1 Entropy and heat 14
4.2 An eight-state instance 14
4.3 No intrinsic joule value 14

5 Windows, Pricing and Narrowing **16**

6 An Example Machine: The Small Machine **18**

7 Thiele Completeness **20**
7.1 The window theorem 21
7.2 Every model of computation 22
7.3 Composition 22
7.4 Each clause is needed 23

8 The Universal Machine **24**
8.1 The universal host 25
8.2 U 25
8.3 The priced host and U_P 25
8.4 A verified compiler and the extracted machines 26

9 Recursion and Undecidability **27**
9.1 The two-counter machine itself 28

10 The Record Axis **29**
10.1 Growing and probabilistic records 29
10.2 Bases 30
10.3 The record over any order 30

11 CHSH and Tsirelson **31**
11.1 The check as a property of the multi-register machine 31

12 The Physical Boundary **33**

13 External Systems and the Pointer Thesis **34**
13.1 The pointer criterion 34

A Proof File Index **36**

B Epistemological Classification **39**

Chapter 1

Introduction

The Thiele Machine describes a computation by what it reads and writes, by a record of what the computation established, and by a ledger of what establishing it cost, with a rule tying the last two together: the step that raises the record pays. The chapters below state that rule, derive its consequences, show which machines carry it, and prove where a window that hides the record and the ledger loses information.

Chapter 3 states the model, its toll and the universal floor. Chapter 4 derives the toll on finite machines from the price of merging states and checks which premises are needed. Chapter 5 states what a window can decide and price. Chapter 6 defines the small machine. Chapter 7 defines Thiele completeness and certified Thiele completeness, states the window theorem for each, lifts any universal base of any model of computation to a certified Thiele-complete machine, composes certified Thiele-complete machines, and shows each clause is needed. Chapter 8 states the universal host, the universal machine U, the priced host's program U_P, a verified compiler onto these machines and the extraction to OCaml. Chapter 9 states the recursion and undecidability results, for L and for the two-counter machine itself. Chapter 10 states the record axis over any base and the latch decomposition of growing records. Chapter 11 states the CHSH and Tsirelson mathematics. Chapter 12 states the physical boundary. Chapter 13 states the models of external systems and the pointer thesis.

Chapter 2

Notation and Conventions

- $\mathbb{N}$ is the natural numbers, $\mathbb{B} = \{\text{false}, \text{true}\}$, $\mathbb{R}$ the reals.
- A *trace* is a finite list of instructions. $\text{run}(\tau, s)$ folds the step function over τ from s.
- $(R, \leq)$ is the preorder of records and $\text{rec} : S \to R$ the record. In Coq it is `ax_rec` on an `AxSys`.
- P is the record on the two-point order, a reading from states to $\mathbb{B}$. In Coq it is `cs_cert` on a `CertificationSystem`.
- $c(i) \in \mathbb{N}$ is the cost of instruction i; $c(\tau)$ is the sum over a trace.
- μ is the ledger. Where a state carries it, conservation means $\mu(\text{step}(s, i)) = \mu(s) + c(i)$.
- A *window* is a function from states to observations.

Chapter 3

Thiele Machines and the Universal Floor

Definition 3.1 (Thiele machine; `AxSys` in Coq)**.** A Thiele machine is a tuple $(S, I, \text{step}, c, \text{rec})$ with $\text{step} : S \times I \to S$, $c : I \to \mathbb{N}$ and $\text{rec} : S \to R$, where $(R, \leq)$ is a preorder given by a Boolean test. The record is a position in that order. A step from s by i *leaves the down-set* when $\text{rec}(\text{step}(s, i)) \not\leq \text{rec}(s)$. The one rule is the toll, first named No Free Insight:

$$\textbf{A2:} \quad \forall s\, i,\ \text{rec}(\text{step}(s, i)) \not\leq \text{rec}(s) \implies c(i) \geq 1.$$

Coq: the record `AxSys` in `coq/kernel/foundation/AxCore.v`, with the toll as `ax_a2`. The order carries no further assumption: not antisymmetry, not growth of the record along steps, not finiteness. Neither permanence nor an internal ledger is part of the definition; the results that use them say so.

Computation as the special case. Forget c and rec and what remains is (S, I, step), a deterministic transition system. Every such system is a Thiele machine: give it a constant record and price zero on every instruction. No step leaves the down-set, so A2 holds. This is immediate from the definition and has no separate Coq counterpart. The theorems of this reference are about what that forgetting throws away.

The one-bit instance. Take $R = \mathbb{B}$ with false $<$ true (`two_pre`). The record is a reading $P : S \to \mathbb{B}$, leaving the down-set is a false-to-true crossing, and A2 reads

$$\forall s\, i,\ P(s) = \text{false} \wedge P(\text{step}(s, i)) = \text{true} \implies c(i) \geq 1.$$

Coq: the record `CertificationSystem`, with this instance of A2 as its field `cs_cert_costs`; `cs_axsys` and `axsys_cs` translate each way. The name of that record does not restrict the interpretation of P: it is an arbitrary Boolean reading, and neither permanence nor the occurrence of both values is required. Most results below are stated on this instance, and each says so by writing P.

Theorem 3.1 (The floor is the toll)**.** ***(S)*** *For every Thiele machine, every run whose final record is not at or below its starting record costs at least* 1 *exactly when A2 holds (`ax_floor_iff_a2`), and under A2 every run costs at least the number of its steps that leave the down-set (`ax_cost_ge_exits`). When the record never drops along a step, leaving the down-set is a strict rise (`ax_exit_is_strict_rise`).*

Proof. A one-step run is a step, so the floor gives the toll. For the converse, induct on the run. If the first step stays in the down-set, the rest of the run ends outside the down-set of its own start, by transitivity of the order, and the induction hypothesis pays. If the first step leaves the down-set, A2 pays. Counting exits is the same induction. □

Theorem 3.2 (Thresholds are one-bit machines)**.** ***(S)*** *For every point a of R, reading the record only as the bit "$a \le \mathrm{rec}(s)$" gives a one-bit Thiele machine, and it obeys its toll whenever the original does (`ax_view_a2`); the original obeys A2 exactly when every such threshold does (`ax_a2_iff_views`). When the order is antisymmetric, the thresholds determine the record (`bp_thresholds_determine`). The one-bit floor of Theorem 3.9 is the two-point instance (`two_point_nfi`), and the four clauses of certified Thiele completeness in Chapter 7 are, one for one, the clauses of the same definition stated on the axis at the two-point order (`ax_tc_two_point_iff`).*

Sliced interface. On the one-bit instance, a sliced Thiele machine additionally has a surjection $\pi : S \to B$ and a partition $I = I_H \sqcup I_V$. Write $F_i(s) = \mathrm{step}(s, i)$. For every $i \in I_H$ there is $h_i : B \to B$ with

$$\pi(F_i(s)) = h_i(\pi(s)), \qquad P(F_i(s)) = P(s), \qquad c(i) = 0.$$

For every $v \in I_V$, require $\pi(F_v(s)) = \pi(s)$ and $P(s) \le P(F_v(s))$, ordering false below true. A2 still applies. These equations hold on all states of the stated domain; they imply permanence. They do not require a product state space or a universal base.

For a non-certification instance, use $S = \mathbb{Z}_4 \times \{0, 1, 2\}$, $\pi(b, k) = b$ and $P(b, k) = [k = 2]$. The horizontal instruction $R(b, k) = (b + 1 \bmod 4, k)$ costs zero; the vertical instruction $V(b, k) = (b, \min(k+1, 2))$ costs one. Projection and reading preservation for R are immediate; V preserves the projection, never lowers the reading, and pays one on its crossing from $k = 1$ to $k = 2$. It also costs one at $k = 2$.

Structural generalization. For any deterministic base (B, J, d) and event $E : B \to \mathbb{B}$, put $S = B \times \mathbb{B} \times \mathbb{N}$. Computing sends (b, r, l) to $(d(b, j), r, l)$ at price zero; a mark sends it to $(b, r \vee E(b), l+1)$ at price one. Projection to B has the section $b \mapsto (b, \mathrm{false}, 0)$ and commutes with every computing run. An event witness supplies a same-view pair with different readings. This preserves base transitions, not a base's previous price or reading. Coq: `coq/audit/StructuralBridge.v`; its specialization to the repository's tape model is `coq/audit/TuringStructuralBridge.v`. No effectivity is assumed by this structural theorem. Ordinary encodings can retain the added coordinates when effective presentations are supplied. A Boolean valuation need not itself be an autonomous two-state quotient: `coq/audit/ValuationBoundary.v` gives a counterexample.

3.1 Free moves, least prices and the observable lift

Theorem 3.3 (All readings and free moves)**.** ***(S)*** *For a state type with decidable equality, fixed transitions and natural instruction prices, A2 holds for every Boolean reading if and only if every zero-cost instruction is the identity on all states. Coq: `every_reading_toll_iff_free_moves_are_identity`.*

Proof. If a free move changes s to t, the decidable reading $[x = t]$ is false at s and true at t, contradicting A2. Conversely an identity move cannot raise any reading, and every positive natural price is at least one. □

This distinguishes a chosen priced reading from arbitrary state predicates; it does not select an autonomous base projection or a partition of moves.

Theorem 3.4 (Free reading preservation)**.** ***(S)*** *Under A2 and global permanence, every zero-cost instruction preserves the reading. With the other sliced-interface laws fixed, global permanence can replace horizontal reading preservation and vertical monotonicity. Coq: `free_reading_invariance`.*

Proof. A free move cannot raise the reading by A2 and cannot lower it by permanence. Conversely, horizontal preservation and vertical monotonicity ensure that no instruction lowers the reading, since the two classes exhaust the instruction set. □

The existing `SlicedInterface` uses the latter presentation; its derived permanence is not an independent premise for removing a field.

Theorem 3.5 (Least instruction toll)**.** ***(S)*** *Fix the transition and reading, and supply a finite list covering all states. For each instruction i, set $p(i) = 1$ if some state has a false-to-true crossing under i, and $p(i) = 0$ otherwise. Then A2 holds for a natural schedule c exactly when $p(i) \le c(i)$ for every i. The same equivalence holds for the global finite-trace crossing floor. Consequently p is the unique pointwise least valid schedule. Coq:* `toll_exactly_dominates_least_price`*;* `trace_floor_exactly_dominates_least_price`*;* `unique_least_valid_price`*.*

Proof. Search the covering list. A crossing witness forces every valid instruction price to be at least one. Without a witness, zero meets every demand for that instruction. The resulting schedule is valid and no larger than any valid schedule. Antisymmetry proves uniqueness. A2 implies the trace floor by the first crossing, while one-step traces give its converse. □

The finite list supplies an effective search; it is not a minimization algorithm for arbitrary infinite systems.

Theorem 3.6 (The coarsest observational identification)**.** ***(S)*** *For a total deterministic system with specified observation o, let $s \sim t$ mean $o(\mathrm{run}(w, s)) = o(\mathrm{run}(w, t))$ for every finite word w, including the empty word. This is the unique coarsest observation-preserving, step-stable equivalence. Any deterministic implementation preserving steps and observations must separate states that some future observation separates. The coarsest relations are pointwise equivalent. Coq:* `future_equivalence_coarsest`*,* `coarsest_relation_unique`*,* `encoding_cannot_merge_distinguishable_states`*.*

Proof. Equality for each word gives the equivalence laws. The empty word gives current-output preservation; prefixing a move gives step stability. Any stable output-preserving relation preserves outputs along a word by induction, so is contained in $\sim$. An implementation merging a separated pair would have to give two answers after the same continuation from one state. □

Uniqueness here is pointwise logical equivalence of relations.

The comparison class. Fix a deterministic base (B, J, d), mark labels K, and events $E : K \times B \to \mathbb{B}$. An implementation has arbitrary state type X, computing and mark operations, view $q : X \to B$, reading $r : X \to \mathbb{B}$ and ledger $\mu : X \to \mathbb{N}$. Computing projects to d and costs zero. Marks fix q, cost one and satisfy the exact latch equation $r(V_k x) = r(x) \vee E(k, q(x))$. Require A2, global permanence and exact accounting $\mu(F_i x) = \mu(x) + c(i)$. An onto view is not required here. The product lift has states (b, r, l), computing rule $(d(b, j), r, l)$ and mark rule $(b, r \vee E(k, b), l + 1)$. It satisfies every law of this class directly.

Theorem 3.7 (Terminal lift and reachable minimal realization)**.** ***(S)*** *Every implementation in this class has a unique coordinate-preserving map to the product lift, namely $x \mapsto (q(x), r(x), \mu(x))$. It commutes with all steps and runs. Its kernel is precisely future observational equivalence for observation (q, r, μ). From a start with reading false and ledger zero, it maps onto all reachable product states from the corresponding start. If reachable implementation states are separated by future observations, this map is injective there, and is the unique start- and step-preserving map on reachable states. Equality of current triples is equivalent to equality of all future observations. Coq:* `canonical_satisfies_contract`*,* `lift_terminal_over_exact_coordinates`*,*

projection_preserves_runs, observable_equivalence_exactly_projection_kernel, reachable_minimal_realization.

Proof. The coordinates force the map. For computing steps the view follows d, the free-reading theorem preserves r, and accounting preserves μ. For marks the exact latch and accounting laws give the product mark rule. Induction gives the run equation. Equal projected triples have equal future triples by deterministic product evolution; conversely the empty word already distinguishes unequal triples. Run any word reaching a product state on the implementation to obtain a preimage. Behavioral separation then turns equal projected triples into equal reachable states. Any map preserving start and steps is forced on an endpoint by a word reaching it. □

This is a bijection on reachable states, not a choice-based construction of an inverse function or an effective minimization algorithm. Observing only (q, r) defines a different equivalence from observing the ledger too.

Histories and the direction of the map. Finite instruction lists, with start [] and steps appending one instruction, have exactly one start- and step-preserving evaluation into any pointed machine: the empty list fixes the start and appending fixes each next value. For the preceding implementation, evaluation followed by projection equals evaluation in the lift (`trace_extension_factorization`). Thus histories map to implementations, and implementations map to the lift. The reverse arrow need not exist. Take a Boolean base toggled by a free move and a mark event always true. Add to the lift a counter increased only by free moves. All contract laws still hold. Two free moves return every lift state to itself, while increasing the extra counter by two. No map from the lift can commute with those free moves, even on reachable states. Coq: `no_map_even_on_reachable_states`.

More generally, give a pointed source machine an access word reaching each state. A start- and step-preserving map into a pointed target exists exactly when equal source endpoints of any two words imply equal target endpoints. Necessity follows by mapping the endpoints. For sufficiency, map a source state to the target endpoint of its access word; the implication makes this independent of all other words reaching it. The empty word and appending a move give the required laws, and reachability forces pointwise uniqueness. Coq: `unique_factorization_iff_kernel_inclusion`.

If two reachable machines have the same initial output behavior and each is separated by future observations, this implication follows by appending all continuations and using target separation. With supplied access words for the first machine, the resulting map preserves outputs and is onto by target reachability and injective by source separation. It is pointwise unique (`minimal_realizations_uniquely_isomorphic`). This is behavioral minimality, not a claim about smallest infinite cardinality.

Theorem 3.8 (Least unit schedule for the lift)**.** ***(S)*** *If every mark event has a base-state witness where it holds, the lift's zero/one schedule is pointwise least among all natural instruction schedules satisfying A2 for its fixed dynamics and reading, over arbitrary base types. Coq: canonical_unit_price_is_least_when_events_possible.*

Proof. Computing preserves the reading, so zero suffices. For each mark take a witness base state with reading false; the mark raises it and forces price at least one. The existing zero/one schedule satisfies A2, so is least. □

Witnesses need not be reachable from a selected start. An identically false mark event permits price zero under A2; unit accounting then adds a demand. These characterizations fix the base, events, observations and price unit. They determine the realization relative to that contract.

Cost interfaces. The definition uses instruction costs $c : I \to \mathbb{N}$, so a trace's cost is independent of its initial state. Later source-charge results use a broader interface $a : S \to \mathbb{N}$ for one autonomous step, or $\widetilde{c} : S \times I \to \mathbb{N}$ with instructions. Their trace cost is the sum of charges along the actual run. The same first-crossing proof gives a floor if each crossing costs at least one, but instruction-priced theorems retain their stated signature and hypotheses. In particular, a single instruction with one fixed price cannot charge one at entry and zero at idle. The latter construction is the source-charge example `paid_entry_free_idle` in `minimal/AccountingBridge.v`.

Theorem 3.9 (The universal floor)**.** ***(S)*** *In every one-bit Thiele machine, if $P(s_0) =$ false and $P(\mathrm{run}(\tau, s_0)) =$ true, then $c(\tau) \geq 1$. Coq: `universal_nfi_any_substrate`; it is also the two-point instance of the floor over any order (`two_point_nfi`).*

Proof. Induction on τ. Since P is false at the start and true at the end, some step in τ takes it from false to true. By A2 that step costs at least 1; every other step costs at least 0. □

Theorem 3.10 (Simulating systems)**.** ***(S)*** *Let a base Thiele machine be simulated by a host: an embedding of base states into host states, a decoding of each base instruction into a list of host instructions, a commuting step law, and certification reflected by the embedding (record `SimulatingCertificationSystem`). A base trace from uncertified to certified costs at least 1 in the base, the host run of its decoded trace ends certified, and that host run costs at least 1. Coq: `host_represents_simulating_cert_system`.*

The commuting law compares endpoints of complete translated instruction blocks. The host's cost floor uses its own toll axiom. This theorem alone neither constrains intermediate block states nor establishes an earned history for arbitrary host programs. Such guarantees require preservation over the host's permitted operations and initial states. The checked counterexamples in `coq/audit/MacroBoundary.v` and `coq/audit/PaidBypassBoundary.v` separate these claims; the latter extends the host with a hypothetical instruction, not an instruction of the original host.

Theorem 3.11 (Quantitative form)**.** ***(S)*** *Extend a Thiele machine with a witness $w : S \to \mathbb{N}$ and a threshold k such that each step raises w by at most its cost and every certified state has $w \geq k$ (record `QuantitativeCertificationSystem`). A trace from a state with $w = 0$ to a certified state costs at least k. Coq: `universal_nfi_quantitative`.*

Theorem 3.12 (A2 is a restriction)**.** ***(S)*** *A cost-bearing system without A2 can switch certification on at zero cost, and A2 excludes exactly that. Coq: `honest_cost_tracking_strict_restriction`, `free_forgery_violates_A2`.*

3.2 A2 among cost frameworks

Theorem 3.13 (The toll as amortized cost)**.** ***(S)*** *A step system satisfies A2 if and only if every step has non-negative amortized cost under the potential that is 1 on uncertified states and 0 on certified ones; the universal floor is then the potential method's telescoping bound. Coq: `a2_iff_nonnegative_amortized_cost`, `nfi_by_potential`. Under A2 the number of certification flips along a trace is at most its cost (`flips_le_cost`), and A2 together with the matching upper-bound condition is equivalent to exact pricing of a permanent certificate (`a2_and_aara_iff_exact`).*

These are local correspondences. The cost-framework families they touch (graded monads, amortized resource analysis, cost translations, linear logic) are not embedded here.

3.3 Several floors and one charge

Theorem 3.14 (Joint floors)**.** ***(S)*** *For a finite family of events, the least cost schedule meeting every unit floor charges one on a step where any event occurs and zero otherwise. Coq:* `joint_floor_is_least`*. On pairs of Booleans, two independent coordinates that both rise in one step are met by a single charge of* 1 *(*`independent_coordinates_joint_change_costs_one`*).*

Independent state coordinates do not by themselves force additive cost.

Theorem 3.15 (Exact pricing)**.** ***(S)*** *For trusted local-predicate pricing systems, a schedule meets the certification floor and never overcharges exactly when it charges on precisely the certification flips, one unit each, and nothing elsewhere. Coq:* `exact_commitment_pricing_characterization`*.*

Chapter 4

Finite Machines and the Price of Merging

Fix a state type S with a finite enumeration, instructions I, a step function and a reading P. The certificate is *permanent* when no step switches it off. An instruction is *injective* when its step function is.

Theorem 4.1 (A permanent flip merges)**.** ***(S)*** *On a finite state space with a permanent certificate, an instruction that switches the certificate on at some state is not injective. Coq:* `permanent_flip_is_not_injective`*.*

Proof. If the instruction were injective it would map the certified states injectively into the certified states, so onto them, leaving no certified image for the uncertified state it switches on. □

This asserts non-injectivity of the instruction, not a collision at the entry destination. For $u \notin H$, $f(u) \in H$, and $f(H) \subseteq H$ with $|H| = K$, the path $u \to h_1 \to \cdots \to h_K \to h_K$ delays the orbit's collision until the $(K+1)$st transition. The entry shares its destination with a source in H exactly when $f(u) \in f(H)$. Surjectivity of $f|_H$ is sufficient, not necessary, for this condition at a particular entry. Instruction pricing charges every use of a non-injective instruction under the merge-pricing premise, regardless of where its collision occurs.

Theorem 4.2 (A2 from merge pricing)**.** ***(S)*** *On a finite state space with a permanent certificate, if every non-injective instruction costs at least* 1*, A2 holds. Coq:* `a2_from_merging_price_and_permanence`*; the trace form is* `permanent_certification_trace_floor`*, and an honest erasure account gives A2 too (*`honest_erasure_accounting_implies_a2`*).*

Each premise is needed. Unbounded history walks past finiteness by paying in fresh states, a new one every step (`unbounded_history_escapes`); a revocable certificate keeps nothing (`revocable_certificate_escapes`); and an unpriced merge is a price list that skips the merge (`free_merge_escapes`). On any state space, a raise that merges nothing sends the certified states one-to-one into a proper part of themselves, so it needs infinitely many of them.

Theorem 4.3 (The squeeze bound)**.** ***(S)*** *On a finite state space with a permanent certificate, with m certified states and k uncertified states that one instruction switches on, the instruction collapses at least k states (*`permanent_flips_collapse_at_least`*). Priced one unit per halving of the number of distinct states, its cost c satisfies $m + k \le 2^c m$, so, when $m \ge 1$, $c \ge \log_2((m+k)/m)$; in rounded-up logarithms, $\lceil \log_2(m+k) \rceil \le c + \lceil \log_2 m \rceil$. Coq:* `permanent_flips_compression_bound`*,* `permanent_flips_log_bound`*.*

These bounds are tight, and A2 is equivalent to the floor. For every system, A2 holds exactly when every run from an uncertified state to a certified one costs at least 1 (`nec_f_floor_iff_a2`). For every $m \geq 1$ and c, a grid machine with m certified states and one move of cost c flips $2^c m - m$ states, so the bound $m + k \leq 2^c m$ of the theorem above is met with equality (`nec_f_squeeze_tight`). Its rounded-logarithm form is strictly weaker: $m = 3$, $k = 1$ and cost 0 meet the rounded form and fail $m + k \leq 2^c m$ (`nec_f_rounded_log_form_weaker`).

Theorem 4.4 (Which flips are forced)**.** ***(S)*** *When instructions have decidable equality, over all schedules that price merges, an instruction is forced to have positive price if and only if it merges two states (`forced_priced_iff_merges`). An instruction that switches a reading on either merges or switches it off somewhere else (`flip_merges_or_revokes`). So every permanent-record write is forced, and the only flips a merge-priced schedule can leave unpriced are revocable ones, which keep nothing.*

Theorem 4.5 (Refutation)**.** ***(R)*** *"The forced events are exactly the permanent-record writes" is false: a three-state instruction flips, revokes, and merges at once. Coq: `forced_price_without_permanent_record`.*

Theorem 4.6 (Merge, or a fresh state every step: the dichotomy)**.** ***(S)*** *On any state space, let a reading be permanent under a step f, and take a run $x_0, f(x_0), f(f(x_0)), \ldots$ in which the reading goes from false at step r to true at step $r + 1$. If no two states among the run's first n steps merge under f, those states are pairwise distinct; so on a finite state space of M elements, a run that keeps a record and never merges has fewer than M steps. In particular, for an injective move F that switches the reading on at s, the iterates of s are pairwise distinct and no finite duplicate-free list contains them all: a permanent flip either merges or visits a fresh state every time. Coq: `run_merges_or_fresh`, `run_bounded_by_room`; `permanent_flip_iterates_distinct`, `permanent_flip_no_finite_list`.*

Theorem 4.7 (Retention bound, and its exactness)**.** ***(S)*** *Let a finite implementation $F : X \to X$ of a reset carry an abstraction q that sends every image of F to a fixed abstract state q_0, with a representative of q_0 and representatives of $N - 1$ further, pairwise distinct abstract states, and let F be injective on the states reached from the representatives in fewer than T applications. Then $|X| \geq 1 + (T + 1)(N - 1)$; for $N \geq 2$, the history is therefore kept through at most $\lfloor (|X| - N)/(N - 1) \rfloor$ resets. For $N = 1$ the bound is only $|X| \geq 1$ and gives no finite retention horizon. The bound is attained: one fixed state and $N - 1$ paths of $T + 1$ states, with F walking each path and sending its last state to the fixed one, has exactly $1 + (T + 1)(N - 1)$ states, is closed under F, meets the reset and representative premises, is injective through T repetitions, and, when $N \geq 2$, merges on the next. Coq: `retention_bound`, `retention_horizon`; `tight_exact`, `tight_closed`, `tight_retain`, `tight_merges_next`.*

Theorem 4.8 (The step law)**.** ***(S)*** *For any step f on any type with decidable equality and any start, every step of the run before its first revisit is fresh (the new state equals no earlier state of the run); the first revisit lands on the start or is a merge (it equals an earlier state x_j with $j \geq 1$ and the state before x_j differs from the current state, so two distinct states share an f-image); and every later step repeats a step the run already took, so the run is periodic and merges nothing new. Coq: `step_law`, `first_revisit_start_or_merge`, `later_steps_repeat`, `merge_is_collision`, `revisit_means_loop`.*

Theorem 4.9 (Refutation: the halting hole)**.** ***(R)*** *"Every raise of a permanent reading on finitely many states is a merge" is false for partial steps: on two states, $a \mapsto b$ with b having no successor raises the reading with no shared image. Totalising the halt ($b \mapsto b$) makes the raise a merge. Coq: `partial_raises_without_merge`, `totalised_halt_is_merge`.*

Theorem 4.10 (The write of a cell, and the fresh states it takes to avoid the merge)**.** ***(S)*** *On the states blank, zero, one with reading "definite," the write of a 0 is permanent,*

raises the reading from blank, and merges blank with zero. A fourth state remembering that the cell was blank does not avoid the merge: the write on four states still merges two distinct states. Only endlessly many fresh states avoid it, a new one every step: on the natural numbers the write "add one" is injective and never returns to 0; on the integers "add one" is a bijection, raises "at least one" from 0, keeps it, and after $k > 0$ steps is away from its start. Coq: `write_is_permanent`, `write_raises`, `write_is_a_merge`, `four_state_escape_fails`, `tape_write_injective`, `tape_write_never_returns`, `shift_bijective`, `shift_raises_and_keeps`, `shift_never_returns`.

Theorem 4.11 (No state space codes all its own readings; the regress has a price)**.** ***(S)*** *For any type X and any coding $X \to (X \to \mathrm{bool})$, some reading $X \to \mathrm{bool}$ is coded by no state (Cantor). With the entitlement count, a record that narrows N candidates to one costs at least $\lceil \log_2 N \rceil$ units, and for every m there is a candidate count whose rounded-up logarithm is at least m. Coq:* `no_space_holds_its_own_readings`, `regress_price_unbounded`; *the count itself is* `ent_cs_run_entitlement`.

Theorem 4.12 (Nondeterministic merge)**.** ***(S)*** *Let a step relation on a finite duplicate-free list of states have a duplicate-free set Y of yes-states that is closed and total (every yes-state has a chosen successor in Y), and let some state outside Y relate to a state of Y. Then some state of Y has two distinct predecessors. Coq:* `nondet_merge`.

Theorem 4.13 (Reversible keeps nothing)**.** ***(S)*** *For an injective step on a finite duplicate-free list containing every state, the run from any start returns to the start after a positive number of steps, and any reading that is false at the start and never goes from true to false is false at every step. Coq:* `returns_to_start`, `reversible_keeps_nothing`.

Theorem 4.14 (Noisy steps, both directions)**.** ***(S)*** *Let P be a nonnegative kernel on n states with rows summing to 1 and let Y be closed under P. If every column of P sums to 1, then $P(i,j) = 0$ for every $i \notin Y$, $j \in Y$. If some $i \notin Y$, $j \in Y$ has $P(i,j) > 0$, then some column sum differs from 1, and the scaled collision weight $\sum_j c_j^2$, with c_j the column sums, strictly exceeds n. For the uniform distribution $u_i = 1/n$, $c_j = n(uP)_j$, so the actual collision probability $\sum_j (uP)_j^2$ exceeds $1/n$, its initial value. Conversely, if every column sums to 1, then for every spread q, $\sum_j (qP)_j^2 \le \sum_i q_i^2$. Over the reals, for a strictly positive spread q on N states, $\sum_i q_i \ln(Nq_i) = \ln N - H(q) \ge 0$, with strict inequality as soon as some $q_i \neq 1/N$. Coq:* `noisy_chairs`, `raise_breaks_uniform`, `colsums_total`, `raise_concentrates`, `uniform_floor`, `raise_concentrates_uniform`, `no_merge_no_concentration`; `gibbs_nonneg`, `gibbs_strict`, `relent_is_entropy_gap` *(the last three over Coq's real numbers).*

Theorem 4.15 (Unitary steps)**.** ***(S)*** *Over the algebraic closure of $\mathbb{Q}$ with complex conjugation, let U be an $n \times n$ matrix with $U^\dagger U = 1$ and P a matrix with $P^2 = P$, $P^\dagger = P$ and $(1-P)\,U\,P = 0$. Then $P\,U\,(1-P) = 0$. Stated over the Mathematical Components library; finite dimension enters once, as "a left inverse of a square matrix is a right inverse." Coq:* `quantum_chairs`.

Theorem 4.16 (Heat against time)**.** ***(S)*** *For $0 < p < 1$ the biased walk's entropy production per tick, $\sigma(p) = (2p-1)\ln\frac{p}{1-p}$, is 0 at $p = \frac{1}{2}$ and strictly positive otherwise. For $0 < v < 1$ and $N \ge 0$, $N \ln\frac{1+v}{1-v} \cdot \frac{N}{v} \ge N^2$. Over Coq's real numbers. Coq:* `heat_at_any_speed`, `no_heat_at_rest`, `heat_time_floor`.

The displayed constant is deliberately weaker than the best bound for this formula. Indeed, the derivative of $\ln((1+v)/(1-v)) - 2v$ is $2v^2/(1-v^2) > 0$ and the expression is zero at $v = 0$. Thus the product is at least $2N^2$, strictly so for $N > 0$. The cited Coq theorem asserts the stated N^2 bound.

Remark 4.1. The monograph also develops four further arguments: one unit per crossing (a run costs at least its number of no-to-yes steps, with no permanence assumed, and that bound

characterizes the toll; Coq: `nec_f_cost_ge_rises`, `nec_f_rises_iff_a2`); the certainty bound (under entropy pricing, the uncertainty about the start equals the part the run destroyed, at most its cost in bits, plus the part the end state still holds); every scale paying the same way (if a coarse description of a machine merges, the fine step merges two described states, leaves the described set, or the described set is infinite); and the device floor (under the stated thermal entropy-balance assumptions, a fixed protocol merging K inputs produces at least $k_B \ln K$ of entropy, at least $k_B T \ln K$ of dissipated work, on its worst input, however slowly it runs and whatever anyone knows).

4.1 Entropy and heat

Theorem 4.17 (Entropy after a permanent step)**.** ***(S)*** *After a permanent step, any distribution on the m certified states and the k flipping ones has Shannon entropy at most $\log_2 m$ (`permanent_step_entropy_ceiling`). From the uniform distribution the step removes at least $\log_2((m+k)/m)$ bits (`permanent_flip_uniform_entropy_drop`). An injective step removes none (`step_entropy_invariant_if_injective`).*

Theorem 4.18 (Heat)**.** ***(C)*** *Given the named premise `landauer_heat`, which states Landauer's principle for a step and a distribution (Rolf Landauer argued at IBM in 1961 that a step that cannot be run backwards requires a minimal heat generation; the premise reads this as mean heat at least $k_B T \ln 2$ per bit of entropy the step removes, for the chosen physical memory and its input distribution, as an average over that preparation rather than a bound on each trial), the instruction has mean heat, when $k_B T \geq 0$, at least $k_B T \ln((m+k)/m)$ from the uniform distribution (`permanent_flip_heat_floor`), and, when $k_B T > 0$, positive heat from any distribution that leaves every certified state and the flipping one possible (`permanent_flip_full_support_heat_positive`). For a state known in advance, the premise is equivalent to heat ≥ 0, so it supplies no positive lower bound; it does not determine actual device heat, which depends on the physical process and preparation, and the ledger toll still charges its unit on a known run, since merge pricing prices the instruction whatever input was prepared (`known_state_flip_forces_no_heat`).*

The monograph adds, on the page and with no Coq counterpart here, a floor that comes from the device rather than the premise. By the identity tying Ladyman's reading of Landauer's principle to Sagawa's, and the second law, a fixed device that merges K inputs produces at least $k_B \ln K$ of entropy, at least $k_B T \ln K$ of dissipated work, on its worst input, however slowly it runs and whatever anyone knows. So on a known start the premise sets no floor, and the merging move still has one on its worst input.

4.2 An eight-state instance

Example 4.1. Four program slots and a certification flag make eight states. An advance step is injective (`fnext_injective`); the certify step and every jump merge states (`fcertify_merges`, `fjump_merges`). The cost rule prices exactly the merges, so A2 holds there as a theorem (`fin_a2_from_merging_price`).

4.3 No intrinsic joule value

Theorem 4.19 (Refutation)**.** ***(R)*** *For every state type and natural-number ledger, the energy assignments at scales 1 and 2 give values 1 and 2 respectively at every state whose ledger is 1. This is conditional on such a state; an identically zero ledger need not yield different energies. The ledger fixes no energy unit. Coq: `mu_has_no_intrinsic_joule_value`. Choosing the scale*

$k_B T \ln 2$ gives one ledger unit that energy by definition (`calibrated_mu_landauer_energy`); the scale is a premise, and nothing here measures it.

Chapter 5

Windows, Pricing and Narrowing

Theorem 5.1 (Window adequacy)**.** ***(S)*** *A query on states can be answered by a function of a window only if it is constant on the window's fibers, the sets of states the window shows the same way (`decoding_requires_fiber_constancy`). Given a representative for every view, fiber constancy is equivalent to factoring through a decoder (`selected_representatives_give_decoder`).*

Adequacy for a supplied operational model. Allow a relation of permitted successors, a chosen state view and required outputs. Supply a representative for each view and a permitted successor for each instruction and underlying state. A deterministic decoder and next-view rule exist exactly when equal views give equal current outputs and every permitted successor under a common instruction has the same view from equal-view sources. This is `operational_adequacy_iff`; it does not establish that the supplied relation describes a physical apparatus, or infer effective choices from existential ones. Observable branching from the same state and instruction excludes an exact deterministic encoding preserving those outcomes (`branching_answers_exclude_exact_deterministic_encoding`).

Selection and finite validation. For the unpointed two-state toggle, every Boolean reading invariant under its state-exchange symmetry is constant (`toggle_symmetry_forces_constant_reading`). Distinguished starts or task meanings are additional structure. For every finite horizon a unit-priced clock with a delayed reset agrees through that horizon with a permanent reading, while both satisfy the actual toll and only one is permanent (`finite_observations_do_not_establish_permanence`). These results constrain conclusions from bare dynamics and bounded tests; they neither refute every physical permanence model nor choose a universal physical description. Sources: `coq/audit/ObservationContract.v` and `coq/audit/IntrinsicBoundary.v`.

Theorem 5.2 (Shadow pricing)**.** ***(S)*** *If a certifying step and a non-certifying step show the same window before and after, no price computed from the window meets the certification floor without charging the non-certifying step. Coq: `shadow_cannot_price_exactly`. A window that shows the reading prices certification exactly (`window_showing_reading_prices_exactly`).*

Theorem 5.3 (Retained history)**.** ***(S)*** *Keeping the previous state in a history makes every deterministic step injective, and the state part of the new pair is still the original step's result, so the observed transition is unchanged. The history takes a fresh state every step; it walks past the finiteness premise, not past what keeping takes. Coq: `retained_history_step_injective`, `history_simulates_observed_step`.*

Theorem 5.4 (Narrowing)**.** ***(S)*** *On a finite machine priced one unit per halving of the number of distinct states, each halving of the machine's own set of possible states along a run is paid for (`run_narrowing_priced_log`). What an observer knows can shrink at zero*

cost: a measurement that copies a hidden bit into a blank display forgets nothing, borrowing the blank display (fresh states), and one admissible merge-priced cost assigns it zero (`observer_narrowing_can_be_free`); wiping the display is a merge and costs at least one (`wipe_costs_at_least_one`). The WIPE charge is an instruction-level bound over all its possible sources. On the particular correlated pair produced by copying, the hidden bit can retain the entire distinction after WIPE. A charge of one is therefore not a claim that this preparation loses one bit of joint entropy, or dissipates one corresponding heat unit.

Chapter 6

An Example Machine: The Small Machine

This chapter builds one machine that meets the definition, to show it can be met. Nothing in the rest of the model depends on its details. The small machine is defined in `minimal/EarnedCore.v` on the Coq standard library alone.

Definition 6.1 (State)**.** A core has two counters $a, b \in \mathbb{N}$, their versions $v_a, v_b \in \mathbb{N}$, a program counter $pc \in \mathbb{N}$ whose lines are numbered from 1 (a value with no instruction, 0 included, is a stop), a table of facts, a commitment channel holding at most one fact, and a trap latch. A fact is a triple (property, counter, version); the properties are "is zero", "is even" and "is at least n", each with a checker `eval` and a meaning `holds` tied by `eval_iff`. A state is a core with a ledger $\mu \in \mathbb{N}$ and a flag $P \in \mathbb{B}$. The table holds at most 16 facts. The start state for (a, b) initializes those counter values without executing a write; both versions are zero, the line is 1, the table and channel are empty, the trap and flag are down, and $\mu = 0$.

Definition 6.2 (Instructions and costs)**.** `INC` c adds one to counter c. `DEC` c j subtracts one and jumps to j if $c > 0$, and otherwise falls through. `HALT` stops. `CHECK` p c evaluates p on counter c and, if it holds and the table has room, writes the fact (p, c, v_c). `COMMIT` p c succeeds only if the fact (p, c, v_c) is in the table, and then sets the channel to it. `CERTIFY` succeeds only if the channel names a fact; success preserves the channel and advances the line. Every failure sets the trap latch, after which no instruction changes anything but the ledger. Every write to a counter adds one to its version and preserves the table and channel. CHECK, COMMIT and CERTIFY cost 1; the others cost 0. The ledger adds the cost of every executed instruction, and the flag rises exactly on a passing CERTIFY.

Theorem 6.1 (Universality)**.** ***(S)*** *The INC/DEC fragment is the two-counter machine step for step, and a program halts on the small machine exactly when it halts as a two-counter program. Coq:* `simulation_run`, `halting_correspondence`*. Its halting problem is undecidable, by reduction from the vendored two-counter result (*`earned_core_halting_undecidable`*). The Turing-universality of two-counter machines is a theorem of Marvin Minsky, who studied how bare a machine can be and still compute what a Turing machine computes (Annals of Mathematics 74, 1961;* Computation: Finite and Infinite Machines*, Prentice-Hall, 1967). It is cited and not re-proved here; the undecidability of two-counter halting used above is checked in Coq in the vendored Coq Library of Undecidability Proofs of Yannick Forster, Dominique Larchey-Wendling and coauthors.*

Theorem 6.2 (The toll)**.** ***(S)*** *The ledger is the exact sum of executed costs (*`mu_conservation_trace`*); only CERTIFY raises the flag (*`only_certify_certifies`*); the flag never drops (*`cert_permanent`*); A2 holds (*`a2`*); and a run from an uncertified state to a certified one pays at least* 1 *(*`nfi_floor`*).*

Theorem 6.3 (Earned provenance)**.** ***(S)*** *From a clean start, every certified run contains a passing CHECK of some claim, then a passing COMMIT of the same claim with that counter untouched in between, then CERTIFY. Coq:* `earned_certification_provenance`*.*

Theorem 6.4 (Checker soundness and no forging)**.** ***(S)*** *On a run from a clean start, a fact whose counter has not been written since its check names a true claim about that counter (*`checker_soundness`*), and every fact in the table was written by a passing CHECK of exactly that claim (*`no_forging`*).*

Theorem 6.5 (Price)**.** ***(S)*** *From a clean start a certified run costs at least* 3 *(*`certified_run_min_cost`*), and a three-instruction program pays exactly* 3 *(*`min_cost_tight`*).*

Theorem 6.6 (A check that can fail)**.** ***(S)*** *The program CHECK "a is zero", COMMIT, CERTIFY certifies from a start with* $a = 0$*, paying exactly* 3*, and from any other start has trapped by its fourth step and never raises the flag, for any number of steps. Coq:* `earned_run_check_can_fail`*,* `earned_run_refused_forever`*.*

Theorem 6.7 (Links)**.** ***(S)*** *The small machine is a Thiele machine with the trace floor (*`earned_core_floor`*), and as a record-carrying machine its record moves as a latch on one base event (*`earned_core_is_latch`*).*

Remark 6.1. `minimal/EarnedGeneric.v` states the same machine over any property language with an exact equality test on properties and an exact checker; sorted lists are one instance (`sorted_cs_sound`). `minimal/EarnedMulti.v` has a counter for every natural number and is the host machine of Chapter 8.

Theorem 6.8 (Each part of the clean start is needed)**.** ***(S)*** *From any state of the small machine, every run that ends certified costs at least* 3 *exactly when the flag is down and either the trap latch is up or the channel is empty and every fact in the table is about an older version of its counter (*`nec_s_floor3_iff`*). Without the empty table some certifying run from a state with an empty channel and the flag down costs less than* 3 *(*`nec_s_min_cost_needs_empty_table`*); without the empty channel some certifying run costs less than* 2 *(*`nec_s_min_cost_needs_empty_channel`*); without the lowered flag some run costs less than* 1 *(*`nec_s_min_cost_needs_flag_down`*). From every clean start with the trap latch down, CHECK, COMMIT, CERTIFY certifies and adds exactly* 3 *to the ledger (*`nec_s_witness_from_every_clean_start`*).*

Chapter 7

Thiele Completeness

Definition 7.1 (Machine)**.** A machine is a state type, a move type, a step, a cost per move and a record $P : S \to \mathbb{B}$ (record `machine`). It is the one-bit instance of the definition in Chapter 3.

The toll and a universal base alone are a low bar: a two-counter machine that charges 1 for every move has a universal base and obeys the toll for any record (`clock_weakly_thiele_complete`).

Definition 7.2 (Thiele complete)**.** A machine is Thiele-complete when it can be read through a universal base (as in clause (a) below, without clean starts), a split of its moves into computing moves and marks, and a ledger, such that: computing moves never change the record and the record once up stays up; every loaded start has the record down; computing moves cost 0, every other move costs exactly 1, and the ledger grows by the cost of each move; and some run ends with the record up from one loaded start and down from another. Nothing in it says what the record is about. Coq: `neutral_complete` in `minimal/NeutralComplete.v`.

Theorem 7.1 (Thiele-complete machines)**.** ***(S)*** *Every Thiele-complete machine obeys the toll and raises its record only by a mark costing exactly* 1 *(`nc_is_weak`, `nc_raise_is_mark`); every run from a state with the record down that ends with it up costs at least* 1 *(`nc_floor_one`); its window hides its record and its ledger, with a gap of one (`nc_hides_record`, `nc_hides_ledger`). A clock and a machine whose record never rises are not Thiele-complete (`clock_not_nc`, `silent_not_nc`), and each clause is needed (`drop_not_nc`, `frozen_not_nc`, `paid_all_not_nc`). Every universal base plus marks on an event that separates two loaded starts is Thiele-complete (`every_base_lifts_neutrally`, `nlift_neutral_complete`). A machine is certified Thiele-complete through an interface exactly when it is Thiele-complete through the same interface read without its claims and its loaded starts are clean, its record is earned and its non-vacuity clause holds (`tc_iff_nc_plus_certification`); the inclusion is strict (`nc_strictly_weaker`).*

Definition 7.3 (Certified Thiele complete)**.** A machine is certified Thiele-complete when its moves can be read as base moves and three record moves, CHECK, COMMIT and CERTIFY of a claim, over a type of claims with a meaning on states, a checker, a notion of the thing a claim is about being unchanged, a set of clean starts and a ledger, such that:

(a) *Universal base.* Every two-counter instruction has a base move that, from every live state, acts on a window of the state exactly as the instruction acts on a two-counter configuration and leaves the state live; every two-counter start has a clean live state showing it (its loaded start), base moves never change the record, and the record once up stays up.

(b) *Earned record.* A clean start has the record down. Every run from a clean start that ends with the record up contains a passing CHECK of some claim, then a COMMIT of the same claim with the thing it is about unchanged in between, then the CERTIFY that raises the record. A passing check means its claim holds, and a claim that holds keeps holding while the thing it is about is unchanged.

(c) *Exact prices.* Base moves cost 0, each record move costs exactly 1, and the ledger grows by the cost of each move.

(d) *Non-vacuity.* Some claim is true at some loaded start and false at another, and the bare chain CHECK, COMMIT, CERTIFY on it raises the record from a loaded start exactly when the claim holds there.

Coq: `thiele_complete` in `minimal/ThieleComplete.v`.

The claim language is fixed first and the definition is taken over it; the interface may not name the meanings, since checker soundness would then hold for nothing.

Theorem 7.2 (Consequences)**.** ***(S)*** *In every certified Thiele-complete machine: the ledger counts the record moves (`ledger_counts_record_moves`); a certificate reached from a clean start costs at least* 3 *(`certificate_costs_three`); the claim such a certificate stands on held when it was checked and still held when it was committed (`Minimal.ThieleComplete.committed_claim_holds`); some check fails (`check_can_fail`); some run certifies (`some_run_certifies`); the move raising the record from a clean start is a CERTIFY (`only_certify_raises`); and some move costs* 0 *(`complete_has_free_move`).*

Theorem 7.3 (Machines that meet it)**.** ***(S)*** *The small machine is certified Thiele-complete (`earned_core_thiele_complete`), and so are the machine over any property language with an exact equality test on properties, an exact checker and a property true of one value and false of another (`earned_generic_thiele_complete`) and the sorted-list machine (`sorted_machine_thiele_complete`).*

Theorem 7.4 (Machines that fail it)**.** ***(R)*** *Every clock-style machine, with any record, has a universal base and obeys the toll, and is neither Thiele-complete nor certified Thiele-complete (`clock_weakly_thiele_complete`, `clock_not_nc`, `clock_not_thiele_complete`); so is a clock whose record is a latch on its configuration (`latch_clock_weakly_thiele_complete`, `latch_clock_not_nc`, `latch_clock_not_thiele_complete`). A machine with a base that costs nothing and one paid move that raises a latch meets clauses (a) and (c), and since that move raises the record from every state, the machine is certified Thiele-complete through no interface (`paid_latch_meets_base_and_toll`, `one_move_record_excluded`, `paid_latch_not_thiele_complete`). A machine whose record never rises meets (a), (b) and (c) and fails (d) (`silent_meets_base_record_toll`, `silent_not_thiele_complete`).*

7.1 The window theorem

Theorem 7.5 (Every certified Thiele-complete machine hides its record)**.** ***(S)*** *For every certified Thiele-complete machine there are a clean start and two runs from it, the second made of base moves only, that end at the same base window; the first ends certified with its ledger at least* 3 *above the start, and the second ends uncertified with its ledger unchanged (`complete_two_runs`). So no function of the base window recovers the record (`complete_hides_record`) or the ledger (`complete_hides_ledger`) on every reachable state (`thiele_complete_hides`).*

Proof. Clause (d) and clause (b) give a clean start and a certifying run. The base is universal, so base moves alone reach the window the certifying run ends at; base moves change neither the record nor the ledger. □

Every certified Thiele-complete machine is a Thiele machine with the trace floor, and its window is blind to its reading (`thiele_complete_floor`, `complete_cs_window_blind`).

7.2 Every model of computation

Theorem 7.6 (Lifting)**.** ***(S)*** *For any machine with a universal base in the sense of clause (a), any claim language over its states that has an exact equality test and a checker proved equal to its meaning, and any fact-table capacity of at least* 1*, if some claim is true at one loaded start and false at another, the machine that adds the earned layer (base moves that raise a version, and CHECK, COMMIT and CERTIFY costing* 1 *each) is certified Thiele-complete (`lift_thiele_complete`). With the window claims (a register of the window is at least* n*, or the program line is* n*) the premise on claims always holds (`lift_window_thiele_complete`). Conversely, every certified Thiele-complete machine has an interface making it certified Thiele-complete whose base moves form a universal base (`lift_reduct_universal`).*

Theorem 7.7 (Limits of lifting)**.** ***(R)*** *A base whose moves have only finitely many different effects at each state (one with finitely many moves, for instance), or one whose step ignores the current state, lifts to no certified Thiele-complete machine under any claim language and capacity through the direct one-move interface (`lift_finite_branching_not_complete`, `lift_stateless_not_complete`). This does not exclude grouping several base steps into one macro move. A one-counter program of* k *instructions started at* n *halts exactly when it has stopped after* $k(n + k + 1)$ *steps, so one-counter halting is decidable (`lift_oc_halts_bound`, `lift_oc_halts_dec`); two-counter halting is undecidable (`lift_cm_halting_undecidable`), and a computable halting-preserving translation of two-counter instances into one-counter instances would make it decidable (`lift_no_oc_translation`).*

Theorem 7.8 (Models of computation)**.** ***(S)*** *A relation on numbers computable in any one of the seven models of the vendored library (the lambda calculus L, the alternate Minsky machines MMA, Turing machines, binary stack machines, Minsky machines MM, FRACTRAN and mu-recursive functions) is computable in all of them (`lift_num_models_agree`). The number-coded two-counter machine has a universal base, its step relation is computable in all seven models, and the window lifts, with a fact table of* 16*, of that machine and of the random-access machine are certified Thiele-complete (`lift_classic_bases`). Over any record that is a preorder with binary joins (Chapter 10), every universal base, with a claim language that has an exact equality test and a proved checker and a fact table with room for one fact, lifts to a machine that is certified Thiele-complete on the axis, given a claim whose point is not at or below the floor and which is true at one load and false at another (`lift_lax_thiele_complete`). When every record is below every other, no machine over that axis is certified Thiele-complete (`lift_ax_indiscrete_no_machine`).*

7.3 Composition

Theorem 7.9 (Products and hand-offs)**.** ***(S)*** *Two machines certified Thiele-complete on the axis give a product, whose moves act on one part each and whose record is the pair of the two records, that is certified Thiele-complete under the product interface, with every certification keeping its own earned chain (`cmpz_prod_tc`, `cmpz_prod_thiele_complete`). The cost of an interleaved run is the cost of its left moves plus the cost of its right moves (`cmpz_cost_prod`), two interleavings of the same moves reach the same state at the same cost (`cmpz_interleaving_invariant`), and with record-preserving simulations as morphisms the product is the categorical product (`cmpz_prod_universal`). Running one machine and handing its two output counters to a second through loading moves that cost nothing gives a certified Thiele-complete machine when both are (`cmpz_seq_tc`), and it computes the second program's run on the first program's output (`cmpz_seq_composes`).*

Theorem 7.10 (Limits of composition)**.** ***(R)*** *From a clean start of the product, every run that raises the AND of the two records makes at least six record moves (`cmpz_and_costs_six`), and*

the point-level interface through "both flags are up" fails non-vacuity (`cmpz_and_not_complete`*). The product of two clocks has a universal base and obeys the toll, and is not certified Thiele-complete (*`cmpz_or_clocks_not_complete`*). A second writer that changes counter A of the small machine without raising its version breaks the earned-record clause (*`cmpz_shared_not_earned`*). Two finite machines that each have an exact shadow price through a window can collide as a pair (*`cmpz_exactness_not_preserved`*). If two load states of the second machine have records with the first not at or below the second, the second machine with loads that cost nothing added fails the toll (*`cmpz_ld_unclean`*).*

Theorem 7.11 (Nesting)**.** ***(S)*** *If Y runs X and Z runs Y by simulation, then Z runs X with the record bit equal at matched points, halting and the rising of the record corresponding, and the surcharge bounds adding (*`cmpz_link_compose`*); a tower of k links adds between the sum of the lower bounds and the sum of the upper bounds (*`cmpz_tower`*). A computably presented machine whose reading starts at no and whose moves cost at most* 1*, run on U_P (Chapter 8), pays exactly* 2 *more once its latch is up (*`cmpz_host_link_exact`*). A record-preserving simulation that reflects the order of records, into a machine that obeys the toll, tracks every run of the guest by a host run costing at least the number of the guest run's exits (*`cmpz_sim_cost_ge_exits`*).*

7.4 Each clause is needed

Theorem 7.12 (Failures of the completeness clauses)**.** ***(R)*** *Take any interface that makes a machine certified Thiele-complete. Charging every move twice its cost keeps clauses (a), (b) and (d), fails (c), and gives a machine that is certified Thiele-complete through no interface (*`nec_t_doubled_cost_meets_all_but_toll`*). A ledger that always reads* 0 *keeps (a), (b) and (d) and fails (c) (*`nec_t_ledgerless_meets_all_but_ledger`*). An extra move ZAP that raises the record from every state at cost* 1*, read as CERTIFY, keeps (a), (c) and (d) and fails (b), and the machine with ZAP is certified Thiele-complete through no interface (*`nec_t_zap_meets_all_but_chain`*,* `nec_t_zap_not_thiele_complete`*). A checker that accepts every claim keeps (a), (c) and (d) and loses checker soundness (*`nec_t_unsound_check_meets_all_but_soundness`*). A machine whose states pair one of six control states with a ledger, with one move that costs nothing and the ordered record moves CHECK, COMMIT and CERTIFY, meets the loose notion, which keeps every clause except the universal base, and is certified Thiele-complete through no interface (*`nec_t_nb_loose_complete`*,* `nec_t_nb_not_thiele_complete`*). For the generic machine, the exact equality test, the exact checker and a property true of one value and false of another each do work. An equality test that calls a different property equal to one the checker accepts on some value, a checker that accepts a property on a value where it does not hold, or a property language in which every property that holds of one value holds of all, each leaves the generic machine with its interface not certified Thiele-complete (*`nec_t_generic_needs_exact_eqb`*,* `nec_t_generic_needs_exact_eval`*,* `nec_t_generic_needs_varying_property`*).*

Chapter 8

The Universal Machine

Definition 8.1 (Universal Thiele machine)**.** Fix a class C of Thiele machines, each with its runs from given start states. A *universal Thiele machine for* C is a Thiele machine H, the host, with one program P fixed for the whole class, a way to load any guest M in C with any start s_0 into a start state of H, and a way to read a guest's state off the host's registers, such that:

(1) H is Thiele-complete, and its computing moves are its counter moves;

(2) the run of P from the loaded state halts exactly when M's run from s_0 stops, and then shows the observable part of M's last state, as the class names it, its reading kept as a latch, and its ledger up to the surcharge;

(3) the run passes, in order, through host states that show the observable part of M's states along its run, at which the host's record equals M's reading kept as a latch and the host's ledger equals M's ledger plus a surcharge, never negative and bounded by a number fixed for the class;

(4) *the rise is the program's*: from every untrapped state, a record instruction selected by P either traps, or else leaves the flag up exactly when it was already up or the current line is marked as raising by P;

(5) H is a *register host*: its counter moves are INC, DEC and HALT on registers, its record moves leave the registers alone and go to the next line or trap, and which move comes next depends only on the line and whether the trap is up;

(6) each guest is loaded with the record down, no trap, a fixed line and the guest written into the registers, and the guest's state is read off the registers in a coding fixed with the class, before any guest is named.

A *certified* universal Thiele machine also has its host certified Thiele-complete with clean loads, and its record, whenever it is up, standing on the host's own check of the guest's own justification, as the class supplies it.

U and $\mathrm{U_P}$ are certified universal Thiele machines, by the theorems of this chapter put together. For U the surcharge is 0, the guest's justification is its chain claim, and the observable part is the guest's two counters; in order U matches everything but the guest's version numbers. For $\mathrm{U_P}$ the surcharge is at most 2 when the guest's reading starts at no and exactly 3 when it starts at yes, the guest's justification is its reading, and the observable part is the guest's whole state. $\mathrm{U_P}$'s program, read on a register host with no claims and no checker, is $\mathrm{U_N}$; it meets the uncertified definition with the same surcharge (`UN_neutral_universal`). Surcharge 0 for every guest whose reading starts at no, across a universal class, is reached on that host by $\mathrm{U_P}$'s compiler with one TICK in place of the three-act chain at the first yes; the monograph argues it, and it isn't formalised.

The Coq predicate `neutral_universal` (`coq/kernel/foundation/NeutralUnivDef.v`), met by U, U_P and U_N (`U_neutral_universal`, `UP_neutral_universal`, `UN_neutral_universal`), is not the definition above: it lets the decoding depend on the guest, and a host blind to its guest meets it on a class whose guests all halt and record alike (`smuggle_universal`). Clause (6), a coding fixed with the class before any guest is named, is argued in words in the monograph; U and U_P read the guest off fixed registers in the guest's own coding.

8.1 The universal host

Theorem 8.1 (The guest mirror and its toll)**.** ***(S)*** *The host of `minimal/UniversalThiele.v` runs any small-machine program as a guest. From a loaded host, or any state where the two agree, its mirror of the guest's flag always equals that flag (`record_agreement`, `hload_agrees`). The mirror rises only in a guest step that passed CERTIFY, charged in that same host step (`no_free_host_certification_step`, `toll_enforced_by_host_step`). The host is a Thiele machine with the trace floor (`host_nfi`).*

On this host, a machine from elsewhere enters through the two-counter encoding, and its computation and the toll are kept. Its own price list is not kept. On U_P, in the section on the priced host below, a computably presented machine keeps its own ledger plus a surcharge.

8.2 U

U is one fixed program of the machine of `minimal/EarnedMulti.v`, with the single property of `minimal/UniversalCodes.v`, made of the small machine's own instruction kinds and with no guest step built into anything (`coq/kernel/foundation/UniversalLayout.v`). For a small-machine program P and a start (x, y), g_m is the guest after m steps and h_n is U after n steps from the loaded state.

Theorem 8.2 (Simulation)**.** ***(S)*** *For every m there is $n \geq m$ with h_n related to g_m by the simulation relation, or both machines have stopped. Coq: `U_simulation`.*

Theorem 8.3 (Halting and output)**.** ***(S)*** *The guest halts if and only if U halts (`universal_halting`); at halting, U's first two registers hold the guest's counters, and the trap latches, ledgers and flags are equal (`universal_output`).*

Theorem 8.4 (The flag is earned again)**.** ***(S)*** *The guest's flag ever rises if and only if U's flag ever rises (`universal_flag_iff`). A raised flag of U stands on U's own passing CHECK, its own COMMIT of the same claim with the slot untouched, and the CERTIFY that raised it; when checked, the slot held the guest's claim and value, and the guest's own run contains its chain on the same claim (`universal_earned`).*

Theorem 8.5 (The ledger)**.** ***(S)*** *At matching points U's ledger equals the guest's, and every ledger of U lies between the guest's ledger at some m and at $m + 1$. Coq: `universal_ledger_exact`.*

Theorem 8.6 (Completeness and undecidability of U)**.** ***(S)*** *The machine U runs on, read with INC and DEC as its base and CHECK, COMMIT and CERTIFY as its record moves, is certified Thiele-complete (`universal_thiele_complete`). U's halting problem, whether U started on the code of P and the inputs x, y ever halts, is undecidable (`interp_halting_undecidable`).*

8.3 The priced host and U_P

The priced host of `minimal/EarnedMultiPriced.v` is the machine with a counter for every natural number and one more paid move, PAY. Its fixed program U_P runs priced guest programs.

Theorem 8.7 (U_P). ***(S)*** *A priced guest halts if and only if U_P run on it halts (`pu_universal_halting`), and the priced host, read with INC and DEC as its base, CHECK, COMMIT and CERTIFY as its record moves and PAY as a check of the empty claim, is certified Thiele-complete (`pu_universal_thiele_complete`). A computably presented machine runs on U_P: U_P halts exactly when the machine does, and at every step before the halt some host step decodes the machine's state, carries a flag equal to the machine's latch and a ledger equal to the machine's ledger plus a surcharge (`presented_universal`). When the machine's reading starts at no, the surcharge is at most 2 (`presented_universal_surcharge_le_two`); when it starts at yes, it is exactly 3 (`cmpz_host_link_start_up`). If the four code-level relations of a presentation (next move, step, cost and reading) are computable in any one of the seven models of computation of the vendored library, the machine has a presentation that runs on U_P this way (`lift_computable_runs_on_U`).*

8.4 A verified compiler and the extracted machines

Theorem 8.8 (The compiler). ***(S)*** *For a source program with variables, addition, truncated subtraction, conditionals, while loops and non-recursive procedures with several results, whose shape a Boolean check decides (`cmp_wf_b_spec`), the program computes y exactly when the fast runner halts with y, exactly when the compiled host program halts with y at the output, exactly when the compiled two-counter guest does, and exactly when U_P running that guest halts with y, the first counter 0, ledger 0 and the flag down (`cmp_final`). If the source program has a run, there is a k such that the runner halts with any fuel of at least k and uses exactly k steps (`cmp_exec_complete`); a runner that does not halt has used all its fuel (`cmp_exec_unhalted`).*

The interpreter, shape check, compiler and runner, and the host programs U and U_P, are extracted to OCaml (`ocaml/CmpExtract.v`, `ocaml/RealizeExtract.v`). The extracted definitions are proved equal to the originals: the extracted instruction list is U (`rlz_host_program_is`), the extracted host state after n steps is the original host's, for U and for U_P (`rlz_host_at_is`, `rlz_phost_at_eq`), and replacing the register storage of a run by a table changes no field of any state (`rlz_host_sched_sound_original`). Extraction maps natural numbers to unbounded integers; the hand-written mapping lines and the two OCaml drivers are trusted and carry no proof. U on a guest that certifies is proved and not run.

Chapter 9

Recursion and Undecidability

Definition 9.1 (Substrate)**.** A substrate supplies states, programs, a partial run function, a ledger that no step decreases, an encoding of programs into states with a decoder, behavioral equivalence (equal runs from every state), a representability predicate on program transformers, and the recursion theorem stated and proved by the logician Stephen Kleene in 1938, for representable transformers, as a field (class `Substrate`). A shortcut predicate on programs respects behavioral equivalence and has one accepted and one rejected canonical program (class `WithShortcutPredicate`).

Theorem 9.1 (Structural undecidability)**.** ***(S)*** *In every substrate with a shortcut predicate, no decider whose flip transformer is representable returns true exactly when the predicate holds. Coq: `structural_shortcut_undecidable`.*

Proof. Let D be such a decider and $f(p)$ the rejected canonical program if $D(p)$ is true and the accepted one otherwise. The recursion theorem gives q behaviorally equivalent to $f(q)$. If $D(q)$ is true, q behaves like the rejected program, so the predicate fails at q; if false, it holds. Either way D is wrong at q. □

The natural-number substrate meets the recursion premise by construction, since its fixed-point program copies the transformer's output (`nat_structural_shortcut_undecidable`).

Theorem 9.2 (L)**.** ***(S)*** *For the weak call-by-value lambda calculus L, defined in `coq/kernel/foundation/LRecursion.v` from its reduction rules, Kleene's second recursion theorem holds (`second_recursion`), every extensional nontrivial property of L terms is undecidable (`L_rice`), and L halting is undecidable (`L_halting_undecidable`). L is the calculus Yannick Forster and Gert Smolka presented as a model of computation in which computability theory can be carried out in Coq (Interactive Theorem Proving, ITP 2017, LNCS 10499). Yannick Forster, Fabian Kunze, Gert Smolka and Maximilian Wuttke later checked in Coq that L and Turing machines simulate each other (Interactive Theorem Proving, ITP 2021). The Turing-completeness of L is not re-proved in this repository's own files: it comes from the models-equivalence theorem of the vendored Coq Library of Undecidability Proofs (`vendor/coq-undecidability/theories/Synthetic/Models_Equivalent.v`), which is checked with this build.*

Theorem 9.3 (The complement of two-counter halting)**.** ***(S)*** *The complement of two-counter halting is undecidable, composed from the vendored library's reductions (`MM2_HALTING_compl_undec`).*

The diagonal uses recursion and behavioral equivalence; cost plays no part. Deciding a predicate over arbitrary programs is a different task from reading one field of a given state.

9.1 The two-counter machine itself

Theorem 9.4 (Rice and Kleene for two counters)**.** ***(S)*** *For any set X of inputs, a property of small-machine programs that cannot tell apart two programs behaving the same on every input in X (same stopping, same counters, trap latch, ledger and flag, and the same shapes of the fact table and channel) is undecidable when it holds of some program and fails of another, and decidable when it holds of every program or of none (`tc_rice`, `tc_rice_dichotomy`). On packed inputs, the powers of* 2*, one program is universal (`tc_universal`), and for every transformation F of programs that some program carries out on program numbers there is a program e with the same packed function as $F(e)$ (`tc_kleene`). On the multi-register host, a property of programs that depends only on the partial function computed is undecidable when it holds of one program and fails of another (`sm_host_rice_fun`), and Kleene's recursion theorem holds for computed partial functions (`sm_kleene`).*

Theorem 9.5 (Where recursion fails)**.** ***(R)*** *When sameness also compares the shapes of the fact table and the channel, some transformation carried out by a program on program numbers has no fixed point, even on packed inputs (`tc_no_fine_kleene`). On plain inputs, the input in counter A and* 0 *in counter B, the recursion theorem fails: some transformation F carried out by a program has no program e equivalent to $F(e)$, computing the same plain function (`tc2_plain_recursion_false`); the proof rests on the fact that no program P multiplies every input by c*(length P)*, the factorial of an explicit bound on the control size of programs of that length, plus* 1 *(`tc2_no_mult`).*

Chapter 10

The Record Axis

Definition 10.1 (Bases and base-driven records)**.** A base is a deterministic record-free machine: states, a next function, initial states and halting (record `BaseMachine`). A record-carrying machine has states, a next function, initial states, a record, a ledger and halting (record `RCM`). A cover maps it onto a base step for step, preserving initial states and halting. The extension is *base-driven* when its record's next value depends only on the base state and itself, the ledger never decreases, A2 holds, the record is permanent, and some reachable step writes it (`HonestBaseExtension`).

Theorem 10.1 (The record axis is a latch)**.** ***(S)*** *Every base-driven record over every base factors as a latch: there is a base event h such that the base state and the record evolve as $(b, r) \mapsto (\mathrm{next}(b), r \vee h(b))$. Coq: `record_axis_is_latch_holds`. Two permanent records factor as two latches, each reading the other (`record_pair_is_two_latches_holds`).*

Theorem 10.2 (Both premises do work)**.** ***(R)*** *A record that can switch back off is not a latch (`toggle_not_latch`). A record switched on by a hidden clock is permanent and not driven by the computation (`clock_record_permanent`, `clock_record_not_driven`).*

Theorem 10.3 (Which machines carry the axis)**.** ***(S)*** *Every base carries it: the base plus a latch on any event it reaches, charging one unit when the latch sets, is a base-driven record (`latch_core_honest`). A reversible base with unbounded memory carries it reversibly (`history_latch_honest`, `history_latch_injective`). With finite memory, a step that writes a permanent record is not injective (`finite_reversible_cannot_write`).*

10.1 Growing and probabilistic records

Theorem 10.4 (Growing records as families of latches)**.** ***(S)*** *A record whose next value is a function of the base state and its current value, that only grows in a partial order, whose ledger charges every strict change, and that changes at some reachable step satisfies a complete family of threshold-latch equations and keeps its price schedule (`growing_record_decomposes_holds`). A monotone chain of pairwise-distinct k-bit vectors has at most $k + 1$ members (`chain_needs_bits_holds`).*

Theorem 10.5 (Limits of one latch and a fixed schedule)**.** ***(R)*** *One Boolean latch does not carry every growing record: a three-value chain over a one-state base is a counterexample (`one_latch_refuted`). In a finite branching model a deterministic latch does not handle branching (`deterministic_latch_handles_branching_refuted`), and equal branch support and charges do not determine the branch weights (`schedule_determines_probabilities_refuted`).*

10.2 Bases

The relation `weak_base_equiv`, agreement on chosen observations and halting with stuttering steps, is an equivalence (`weak_base_equiv_refl_holds`, `weak_base_equiv_sym_holds`, `weak_base_equiv_trans_holds`). The record axis is a latch over the Cook and Reckhow RAM, the unit-cost random-access machine Stephen Cook and Robert Reckhow defined to measure time bounds (Journal of Computer and System Sciences 7, 1973), and over L (`record_axis_is_latch_on_ram_holds`, `record_axis_is_latch_on_l_holds`); these instances follow from the general theorem and do not distinguish the bases. Cost-respecting simulations between different bases are not constructed.

10.3 The record over any order

The definition of Chapter 3 has the record in any preorder, and the theorems there state its floor and its thresholds. In Coq the general machine is an *axis system*. This section states the further results on that axis.

Theorem 10.6 (Latches, merges and windows on the axis)**.** ***(S)*** *A record that never drops and whose next value is a function of the base state and the current record factors through thresholds: some event h makes a at or below the next record exactly when a is at or below the current record or h fires (`ax_decompose`). A repeat-free chain of k-bit words, each word's set bits among the next word's, has at most $k+1$ members, and some chain has exactly $k+1$ (`chain_bits_exact`). With finitely many states and a growing record, every instruction that leaves the down-set from some state merges two states (`ax_growth_exit_merges`), and pricing every merge gives the toll (`ax_a2_from_merge_price`). For finitely many states and moves and decidable observations, some price computed from a window is exact exactly when the window has no collision (`ax_window_exact_iff_no_collision`).*

Theorem 10.7 (Completeness and entropy on the axis)**.** ***(S)*** *An interface makes a machine of Chapter 7 certified Thiele-complete exactly when the same interface, read over the two-point order with false below true, meets the four clauses on the axis (`ax_tc_two_point_iff`). Every certified Thiele-complete axis machine obeys the toll (`ax_tc_a2`), every run from a clean start that ends with the record not at or below the floor makes at least three record moves and raises the ledger by at least 3 (`ax_tc_certificate_three`), and no function of the counter-machine window gives the record (`ax_tc_independence`). On the multi-register host, for every reading of final states into a preorder that respects agreement on everything a program can read, a property of programs that respects behaving alike on the record is undecidable when it holds of some program and fails of another (`ax_rice_record`); some host program has runs whose claim sets climb in m strict steps exactly when $m \le 16$, the size of the fact table (`ax_single_run_boundary`). Under the halving price, on a state space of any size, for a non-negative weighting and any finite list of landing spots, each given with a repeat-free list of exactly the states the move sends to it, the move's conditional entropy loss over those lists is at most their total weight times the move's cost times $\ln 2$ (`ax_inf_loss_bits`).*

Chapter 11

CHSH and Tsirelson

Until the last section, the results in this chapter are mathematics about correlators and trial counts, and no machine is fixed. The bound named for Boris Tsirelson, who showed in 1980 that quantum correlations give a CHSH value of at most $2\sqrt{2}$, appears here as algebra.

Theorem 11.1 (Classical bound)**.** ***(S)*** *A factorizable correlator has $|S| \leq 2$ (`factorizable_CHSH_classical_bound`); trial counts consistent with one deterministic local strategy cannot exceed the classical bound (`chsh_stat_violation_not_local`).*

Theorem 11.2 (Algebraic Tsirelson bound)**.** ***(S)*** *A rational correlator satisfying the absolute-value bounds and four selected 3×3 polynomial minors has $|S| \leq 2\sqrt{2}$ (`algebraically_coherent_tsirelson_general`). The bound also follows from row constraints and Cauchy–Schwarz (`tsirelson_from_row_bounds`) and is reached (`tsirelson_tight`). No Hilbert space is used.*

Theorem 11.3 (The biconditional)**.** ***(S)*** *On the zero-marginal slice, the column-contractivity conditions on $E_{00}, E_{01}, E_{10}, E_{11}$ hold if and only if the zero-marginal 5×5 NPA moment matrix is positive semidefinite. Coq: `column_contractive_iff_npa_psd`.*

Theorem 11.4 (The integer check)**.** ***(S)*** *If the integer column check accepts eight trial counts, the zero-marginal NPA matrix built from their correlators is positive semidefinite (`column_contractive_check_witness_npa_psd`), and a PSD matrix of that form satisfies the Tsirelson bound (`npa_psd_implies_tsirelson_bound`). The check is sound in this direction only; a rejection is not proved to mean the conditions fail.*

Theorem 11.5 (Elliptope)**.** ***(S)*** *Call a correlator $(E_{00}, E_{01}, E_{10}, E_{11})$ elliptope-realizable when some choice of the two within-party cross moments completes the zero-marginal NPA matrix to a positive semidefinite matrix (`elliptope_realizable`). The comments of `coq/kernel/quantum/ElliptopeCompletion.v` read this existential completion as the CHSH correlator quantum set, following Tsirelson's characterization; no theorem here proves that identification. Elliptope-realizable points satisfy the Tsirelson bound (`elliptope_tsirelson`), and the PR box is not elliptope-realizable (`pr_box_not_elliptope`). An integer membership check is sound into realizability (`elliptope_check_full_sound`).*

No theorem here identifies a correlator with a physical experiment or proves that any device produces CHSH-violating statistics.

11.1 The check as a property of the multi-register machine

This section fixes a machine: the multi-register machine of `minimal/EarnedMulti.v`, with a counter for every number and one property that reads a counter as a tally of eight trial counts and that passes exactly when the integer check `small_chsh_check` passes.

Theorem 11.6 (The check on the machine)**.** ***(S)*** *The integer check passes exactly when every question pair was sampled and the pinned* 5×5 *moment matrix is symmetric and positive semidefinite (*`small_chsh_check_iff`*). From a clean start, a raised flag stands on a CHECK, a COMMIT and a CERTIFY of the CHSH claim on one counter, and that counter held a tally passing the check, with* $S^2 \leq 8$ *(*`small_chsh_flag_implies_tsirelson`*). Every passing tally has* $S^2 < 8$ *(*`nec_e_check_strict`*), so none reaches* $2\sqrt{2}$ *(*`nec_e_check_never_tsirelson`*), and for every* $\varepsilon > 0$ *some passing tally has* $S^2 > 8 - \varepsilon$ *(*`nec_e_check_sharp`*). Each of the check's seven integer facts is needed (*`nec_e_check_facts_irredundant`*), and the check refuses the tally of one round per question pair of every deterministic plan (*`nec_e_check_refuses_all_plans`*).*

Chapter 12

The Physical Boundary

Theorem 12.1 (Calorimeter)**.** ***(S)*** *In a two-state calorimeter protocol, a canonical reset from half-excited to ground satisfies a discrete master equation exactly (*`canonical_reset_satisfies_master_equation`*), and at fixed Hamiltonian the bath receives* $\Delta/2$ *(*`canonical_reset_heat_exact`*).*

Theorem 12.2 (Refutation)**.** ***(R)*** *Population dynamics do not fix heat: two gaps with identical dynamics release different heat (*`master_equation_does_not_fix_heat_scale`*).*

Ruling a gap out needs a thermal premise such as local detailed balance, or device evidence. Converting the ledger to energy requires thermal and device premises; the ledger has no intrinsic energy unit (Chapter 4).

Chapter 13

External Systems and the Pointer Thesis

Each model below is narrow. None is a full security, consensus, database or audit-log theorem.

- **RFC 9162 (Certificate Transparency).** The iterative inclusion and consistency checks of the specification reject the boundary cases (`rfc9162_inclusion_boundary_safe`, `rfc9162_consistency_boundary_safe`) and accept the specification's examples under symbolic digests (`rfc9162_example_inclusion_d0`, `rfc9162_example_consistency_4_7`). Collision resistance and signed-tree-head authenticity are cryptographic premises; neither is proved.
- **Proof-carrying code.** A toy checker accepts exactly when the address-bound verification condition holds (`pcc_checker_accepts_iff_vc`). It mirrors the roles in George C. Necula's proof-carrying code (POPL 1997), where a program ships with a proof that it is safe; the full system is outside this model.
- **Trusted Platform Module (TPM) quotes.** A bare sign/verify interface does not imply quote authenticity (`tpm_interface_authenticity_refuted`).
- **Casper FFG.** Casper FFG is the finality gadget Vitalik Buterin and Virgil Griffith proposed for Ethereum in 2017. Two finalized blocks on conflicting branches imply a slashable quorum (`accountable_safety`). This is the accountable-safety theorem of their paper.

The addressed-RAM and reversible-update files prove only their named fragments. The cost-semantics correspondences are not full cost-framework embeddings.

13.1 The pointer criterion

The pointer-observable criterion proposes that the event a step rule is forced to price is singled out by redundant record proliferation across observers. Twelve candidate events over seven observer maps proliferate as stated (`twelve_candidate_measurements_checked`). The MAC-labelled model does not proliferate (`mac_model_not_proliferating`); applying that to a real system depends on the observer model. A two-observer toggle game with consensus, authenticity and coordinator-free updates revokes its event (`toggle_game_refutes_strong_pointer_necessity`), so those properties do not force permanence. Durable views do imply permanence, by making persistence a premise (`durable_consensus_implies_permanence`). The external consequence survey contains five narrow countermodels, each for a point a governing source already addresses, and five stronger

statements without enough protocol or hardware semantics; no surveyed consequence is simultaneously record-axis dependent, novel, and ready for use. The pointer criterion is a thesis: "faithfully model" and "deployed" tie it to the world and cannot be defined, as in the Church–Turing thesis.

Appendix A

Proof File Index

The effective-model comparison also uses the vendor bridge:
`vendor/coq-undecidability/theories/Synthetic/Models_Equivalent.v`.

Results	**File**
The definition, its floor and the one-bit instance	`coq/kernel/foundation/AxCore.v`, `coq/kernel/foundation/AxTwoPoint.v`, `coq/kernel/nfi/UniversalCertificationCost.v`
Structural lift and tape bridge	`coq/audit/StructuralBridge.v`, `coq/audit/TuringStructuralBridge.v`, `coq/audit/ValuationBoundary.v`
Free moves and least instruction toll	`coq/audit/LeastToll.v`
Future observations and unique realizations	`coq/audit/CanonicalObservation.v`, `coq/audit/ObservationContract.v`
Histories, terminal lift and minimality	`coq/audit/TraceFactorization.v`, `coq/audit/LiftTerminal.v`, `coq/audit/LiftProposalBoundary.v`
Simulation and provenance boundaries	`coq/audit/MacroBoundary.v`, `coq/audit/PaidBypassBoundary.v`, `coq/audit/ConcreteClosureBridge.v`
Source charges and exact accounting	`minimal/AccountingBridge.v`
Generic and multi-register machines	`minimal/EarnedGeneric.v`, `minimal/EarnedMulti.v`, `minimal/EarnedMultiPriced.v`
Universal coding and program layout	`minimal/UniversalCodes.v`, `coq/kernel/foundation/UniversalLayout.v`
Fresh states and sharp retention	`minimal/HeatOrRoom.v`, `minimal/SharpHost.v`
First revisit, halting and kept distinctions	`minimal/StepLaw.v`, `minimal/HaltingHole.v`, `minimal/KeptDistinction.v`
Nondeterministic and reversible steps	`minimal/NondetChairs.v`, `minimal/Recurrence.v`
Stochastic concentration and entropy	`minimal/NoisyChairs.v`, `minimal/GibbsGap.v`
Unitary steps and biased-walk time bound	`minimal/QuantumChairs.v`, `minimal/BrownianComputer.v`
Universal floor, simulating systems	`coq/kernel/nfi/UniversalCertificationCost.v`, `coq/kernel/nfi/HonestCostTracking.v`
Quantitative form	`coq/kernel/nfi/QuantitativeNoFI.v`
Cost frameworks	`coq/kernel/nfi/CostSemanticsComparison.v`, `coq/kernel/nfi/CostFrameworks.v`
Exact pricing	`coq/kernel/nfi/CommitmentPredicateAdequacy.v`
Permanent certificates, merge pricing	`coq/kernel/nfi/PermanentCertification.v`, `coq/kernel/nfi/PermanentRecordPricing.v`
Which finite premises are needed	`coq/kernel/foundation/NecFFloor.v`, `coq/kernel/foundation/NecFSqueeze.v`
Entropy and heat	`coq/kernel/nfi/PermanentCertificationEntropy.v`
Eight-state instance	`coq/kernel/nfi/FiniteCertMachine.v`

No intrinsic joule value	coq/kernel/nfi/PricingPhysicsAudit.v
Shadow pricing	coq/kernel/nfi/ShadowPricing.v
Narrowing	coq/kernel/nfi/KnowledgeNarrowing.v
Window adequacy, joint floors	coq/kernel/frontier/ObservationPolicy.v
The small machine	minimal/EarnedCore.v, coq/kernel/foundation/EarnedCoreLinks.v, minimal/NecSClean.v
Thiele completeness	minimal/NeutralComplete.v, minimal/NeutralMore.v
Certified Thiele completeness	minimal/ThieleComplete.v, coq/kernel/foundation/EarnedGenericLinks.v
The window theorem	minimal/ThieleCompleteWindow.v
Every model of computation	minimal/LiftCore.v, minimal/LiftConverse.v, minimal/LiftOneCounter.v, coq/kernel/foundation/LiftModels.v, coq/kernel/foundation/LiftModelsAll.v, coq/kernel/foundation/LiftAxis.v, coq/kernel/foundation/LiftHeadline.v
Composition	coq/kernel/foundation/CzProd.v, coq/kernel/foundation/CzProdTC.v, coq/kernel/foundation/CzCounter.v, coq/kernel/foundation/CzCat.v, coq/kernel/foundation/CzSeq.v, coq/kernel/foundation/CzWin.v, coq/kernel/foundation/CzTower.v, minimal/CzLink.v, minimal/CzShared.v
Each clause is needed	minimal/NecTToll.v, minimal/NecTEarned.v, minimal/NecTLoose.v, minimal/NecTGeneric.v
Universal Thiele machine	coq/kernel/foundation/UMachDef.v, coq/kernel/foundation/NeutralUnivDef.v, coq/kernel/foundation/NeutralUnivU.v, coq/kernel/foundation/NeutralUnivUP.v, coq/kernel/foundation/NeutralUnivShadow.v, coq/kernel/foundation/NeutralUnivSmuggle.v
Universal host	minimal/UniversalThiele.v, coq/kernel/foundation/UniversalThieleLinks.v
U	coq/kernel/foundation/UniversalRun.v, coq/kernel/foundation/UniversalInterpreterLinks.v
U_P and presented machines	coq/kernel/foundation/UniversalPRun.v, coq/kernel/foundation/PresentedUniversal.v
Verified compiler	coq/kernel/foundation/CmpFinal.v, coq/kernel/foundation/CmpRun.v
Extraction to OCaml	ocaml/CmpExtract.v, ocaml/RealizeExtract.v, coq/kernel/foundation/Realize.v, coq/kernel/foundation/RealizePrograms.v, coq/kernel/foundation/RealizePriced.v, coq/kernel/foundation/RealizeCompact.v
Structural undecidability	coq/kernel/nfi/StructuralUndecidability.v, coq/kernel/foundation/NatSubstrateInstance.v, coq/kernel/foundation/Substrate.v
L recursion and Rice	coq/kernel/foundation/LRecursion.v, coq/kernel/foundation/MM2ComplementUndec.v
Two-counter recursion and Rice	coq/kernel/foundation/TcRice.v, coq/kernel/foundation/TcPacked.v, coq/kernel/foundation/TcNoFine.v, coq/kernel/foundation/Tc2Plain.v, minimal/Tc2Mult.v, coq/kernel/foundation/SmKleene.v

Record axis	coq/kernel/foundation/StructuralCore.v, coq/kernel/foundation/StructuralCoreAnyBase.v, coq/kernel/foundation/StructuralRecordAxis.v, coq/kernel/foundation/RecordAxisDiscrimination.v, coq/kernel/foundation/CrossBaseGranularityCore.v, coq/kernel/foundation/CrossBaseGranularity.v, coq/kernel/foundation/CrossBaseGranularityRAM.v, coq/kernel/foundation/CrossBaseGranularityL.v
Latches, merges and windows on the axis	coq/kernel/foundation/AxLatch.v, coq/kernel/foundation/AxLatch2.v, coq/kernel/foundation/AxMerge.v, coq/kernel/foundation/AxWindow.v, coq/kernel/foundation/AxComplete.v, coq/kernel/foundation/AxComplete2.v, coq/kernel/foundation/AxRice.v, coq/kernel/foundation/AxHostClaims.v, coq/kernel/foundation/AxInfinite.v
Growing and probabilistic records	coq/kernel/foundation/GrowingRecord.v, coq/kernel/foundation/ProbabilisticRecord.v
CHSH and Tsirelson	coq/kernel/quantum/CHSHColumnCheck.v, coq/kernel/quantum/TsirelsonGeneral.v, coq/kernel/category/AlgebraicCoherence.v, coq/kernel/quantum/MinorConstraints.v, coq/kernel/quantum/CHSHStatisticalBridge.v, coq/kernel/quantum/TsirelsonFromAlgebra.v, coq/kernel/quantum/ElliptopeCompletion.v, coq/kernel/quantum/ElliptopeGate.v
The check on the multi-register machine	coq/kernel/quantum/SmallChshCheck.v, coq/kernel/quantum/SmallChshMachine.v, coq/kernel/foundation/NecEChsh.v, coq/kernel/foundation/NecEChshInt.v
Calorimeter	coq/kernel/thermodynamic/CalorimeterProtocol.v
External models	coq/kernel/reductions/RFC9162Merkle.v, coq/kernel/reductions/NeculaPCC.v, coq/kernel/reductions/TPMQuoteAuthenticity.v, coq/kernel/reductions/CasperFFG.v
Pointer criterion	coq/kernel/frontier/PointerObservable.v, coq/kernel/frontier/RecordProliferationSurvey.v, coq/kernel/frontier/EcosystemGame.v, coq/kernel/frontier/ PointerObservableCounterexamples.v

Appendix B

Epistemological Classification

Result	Status	Key premise
The definition and the price in the move		
The floor is the toll	**(S)**	A preorder of records; nothing else
Thresholds are one-bit machines	**(S)**	A2; antisymmetry for the thresholds to determine the record
Universal floor	**(S)**	A2 on the two-point order
All readings and free moves	**(S)**	Decidable state equality; one schedule for every Boolean reading
Least instruction price	**(S)**	Fixed dynamics and reading; finite covering list
Least unit lift price	**(S)**	An event witness for every mark; all-state toll
A2 from merge pricing	**(S)**	Finite states, permanent certificate, merge price
Shadow pricing	**(S)**	A colliding pair in the window
What stays		
Free reading preservation	**(S)**	A2 and global permanence
Record axis is a latch	**(S)**	Base-driven record
Earned provenance, checker soundness	**(S)**	Clean start
What a window forgets		
Canonical future equivalence	**(S)**	Fixed deterministic operations and observations
Terminal lift and reachable minimality	**(S)**	Fixed base/events, exact latch, toll, permanence, unit ledger; separation for injectivity
History map in the reverse direction fails	**(R)**	Toggle base with an additional free-turn counter
Window theorem	**(S)**	Thiele completeness (gap of one) or certified Thiele completeness (gap of three)
The room a kept distinction needs		
Merge or fresh states	**(S)**	Total deterministic step, permanent raise
Sharp retention bound	**(S)**	Finite implementation, distinct representatives, reset and injectivity through the specified horizon
Stochastic and unitary holding	**(S)**	Finite stochastic kernel with column constraints, or finite unitary dynamics and invariant projection
Computation as the special case		
Thiele completeness: floor and window	**(S)**	The four clauses of `neutral_complete`
Certified Thiele completeness of the small machine	**(S)**	The four clauses of `thiele_complete`
Clock-style machines fail	**(R)**	The four clauses

Lifting any universal base	**(S)**	A universal base, a claim language with an exact equality test and a proved checker, a fact table with room for one fact, a claim that can come out either way
Composition	**(S)**	Certified Thiele completeness of both parts
Each clause is needed	**(R)**	One clause weakened
U	**(S)**	Small-machine programs
U_P	**(S)**	Priced guests, computably presented machines
Verified compiler	**(S)**	Well-formed source programs
Extraction to OCaml	**(S)**	Hand-written mapping lines and drivers trusted
Structural undecidability	**(S)**	Recursion theorem of the substrate
L recursion and Rice	**(S)**	L's reduction rules
Two-counter Rice and packed Kleene	**(S)**	Behavioral sameness on chosen inputs
Plain-input recursion fails	**(R)**	Inputs in counter A, 0 in counter B
The world		
Biased-walk heat against time	**(S)**	Specified biased-walk entropy formula; positive speed below one
Heat floor	**(C)**	`landauer_heat`, finite states, permanent certificate, the uniform distribution on the states in play
No intrinsic joule value	**(R)**	None beyond the ledger
Algebraic Tsirelson bound	**(S)**	Polynomial minors
Calorimeter heat scale	**(R)**	Fixed dynamics
Pointer criterion	Thesis	Observer model